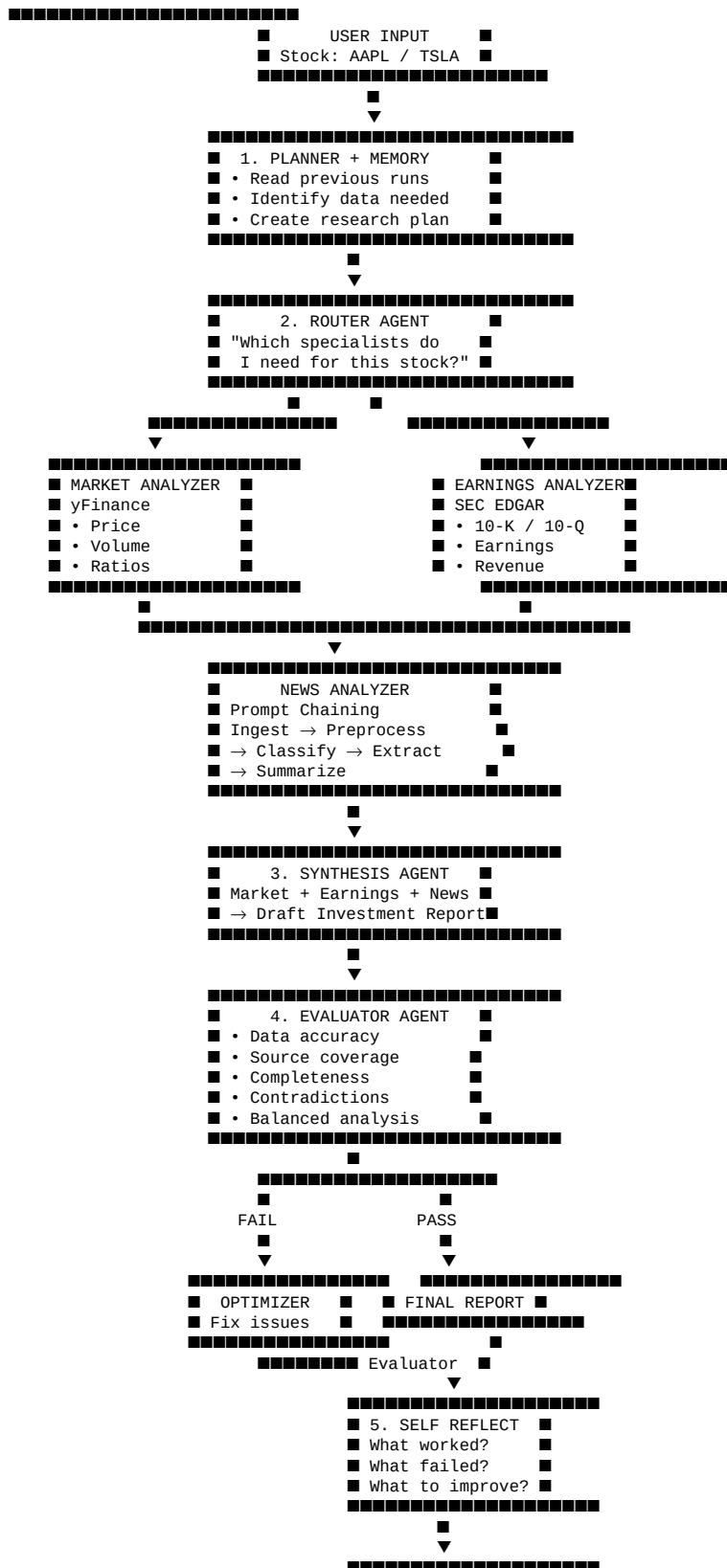
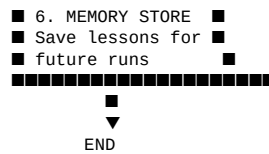


Investment Research Agent — Project Design & Architecture

A suggested architecture and implementation roadmap for the final Agentic AI assignment using LangGraph.

1. High-Level Architecture





2. Three-Layer Design

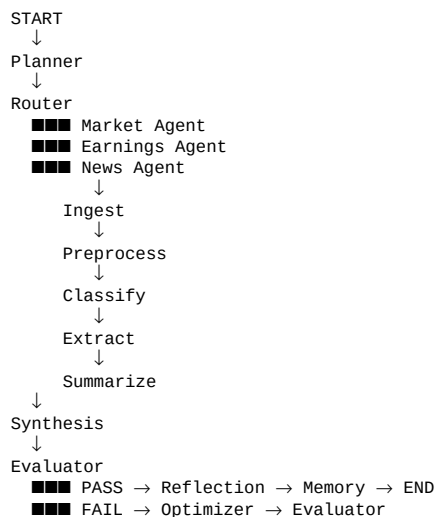
Layer	Responsibilities
User / UI Layer	Stock symbol input; display final investment research report.
LangGraph Agent Layer	Planner, Router, specialist agents, synthesis, evaluator/optimizer, reflection and memory.
Tool Layer	yFinance, SEC EDGAR, News API/Kaggle, FRED, LLM and other data tools.

3. Suggested LangGraph State

```

class ResearchState(TypedDict):
    ticker: str
    research_plan: list
    required_specialists: list
    market_data: dict
    earnings_data: dict
    news_data: list
    economic_data: dict
    cleaned_news: list
    classified_news: list
    extracted_news: list
    news_summary: str
    specialist_findings: dict
    draft_report: str
    evaluation_score: float
    evaluation_feedback: list
    revision_count: int
    previous_memory: list
    new_memory: list
    final_report: str
  
```

4. LangGraph Workflow



5. Mapping Requirements to Implementation

Assignment Requirement	Implementation
Autonomous planning	Planner Agent
Dynamic tool use	Router + tool selection
APIs / datasets	yFinance, SEC, News API/Kaggle, FRED
Self-reflection	Reflection Agent
Learning across runs	JSON or SQLite memory
Prompt Chaining	News ingestion → preprocessing → classification → extraction → summarization

Routing	Market / Earnings / News specialists
Evaluator–Optimizer	Evaluator → Optimizer loop
LangGraph	Main orchestration framework

6. Recommended Design Principle: Don't Make Every Component an LLM

```

BAD:
News → LLM → LLM → LLM → LLM

BETTER:
News API
↓
Python preprocessing
↓
LLM classification
↓
Python extraction / validation
↓
LLM summarization

```

Use deterministic Python for cleaning, deduplication, validation and calculations. Use an LLM where reasoning, classification, synthesis or reflection is genuinely useful. This makes the system clearer and more efficient.

7. Make the Router Genuinely Dynamic

```

User: "Analyze Apple's latest earnings."
↓
Planner
↓
Research tasks:
• latest earnings
• revenue growth
• EPS
• management guidance
• earnings-related news
↓
Specialists:
• Earnings
• News

For another question, the planner may select Market + News instead.

```

8. Evaluator Design

```

Evaluator checks:
✓ Data accuracy
✓ Source coverage
✓ Completeness
✓ Contradictions
✓ Recency
✓ News balance
✓ Output quality

Example structured result:
{
  "passed": false,
  "score": 0.78,
  "issues": [
    "Revenue figure has no source",
    "News analysis lacks bearish evidence"
  ],
  "recommended_actions": [
    "Verify revenue using SEC filing",
    "Retrieve additional evidence"
  ]
}

```

9. Memory / Learning Across Runs

```

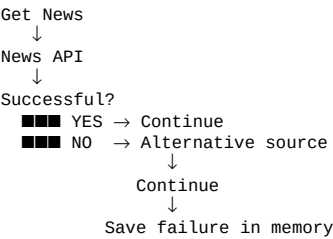
memory/
■■■ research_history.json
■■■ tool_failures.json
■■■ company_notes.json

Example:
{
  "ticker": "AAPL",
  "learnings": [
    "SEC filing retrieval worked",
    "News API returned limited results",
    "yFinance provided sufficient market data",

```

```
        "Prioritize SEC data for earnings questions"
    ]
}
```

10. Tool Fallback



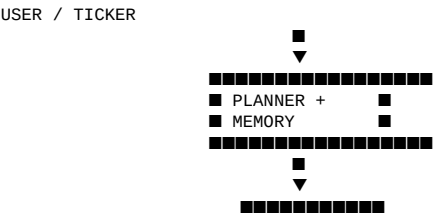
11. Suggested Project Folder Structure

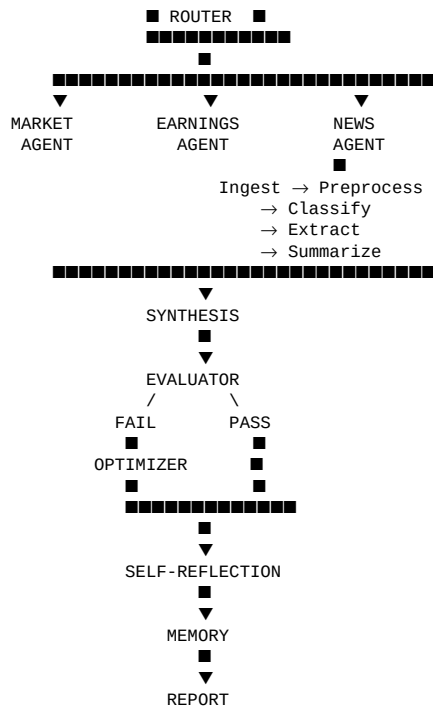
```
investment-research-agent/
├── app.py
├── graph/
│   ├── graph.py
│   └── state.py
├── agents/
│   ├── planner.py
│   ├── router.py
│   ├── market_agent.py
│   ├── earnings_agent.py
│   ├── news_agent.py
│   ├── synthesis_agent.py
│   ├── evaluator.py
│   ├── optimizer.py
│   └── reflection.py
├── tools/
│   ├── yfinance_tool.py
│   ├── sec_tool.py
│   ├── news_tool.py
│   └── fred_tool.py
├── pipelines/
│   └── news_pipeline.py
├── memory/
│   ├── memory_manager.py
│   └── research_memory.json
├── prompts/
│   ├── planner_prompt.txt
│   ├── evaluator_prompt.txt
│   └── synthesis_prompt.txt
├── output/
│   └── reports/
├── requirements.txt
└── README.md
```

12. Recommended Implementation Phases

Phase	Goal
1 — Basic Research	AAPL → yFinance → LLM → basic report
2 — Routing	Planner → Router → Market / Earnings / News
3 — Prompt Chain	Build complete News ingestion → summarization pipeline
4 — Synthesis	Combine specialist findings into one report
5 — Evaluator/Optimizer	Evaluator → FAIL → Optimizer → Evaluator
6 — Memory	Persist run learnings and use them in future planning
7 — Polish	Logging, error handling, source citations, visualizations, README and presentation.

13. Final Architecture for Presentation





Core project idea: build a LangGraph research system that decides what research to perform, selects tools and specialists, processes information through explicit workflow patterns, evaluates its own work, and stores lessons for future runs.